

Ascend C 编程模型：Tensor、缓冲与流水

目 录

1 引言：从 CUDA 经验到 Ascend C	2
2 Tensor 编程模型	2
2.1 LocalTensor 与 GlobalTensor	2
2.2 TPosition：缓冲区的物理位置	2
3 TBuf：静态缓冲区	3
4 TQue：队列与流水	3
5 内存搬运类 API	3
5.1 DataCopy	3
5.2 DataCopyPad	4
6 Vector 计算 API	4
6.1 元素级与逐元素运算	4
6.2 规约运算	4
7 CopyIn, Compute, CopyOut 三段式	4
8 查阅官方 API 参考	5
9 本章你将学会	5
10 要点速查	5
11 小结	6

1 引言：从 CUDA 经验到 Ascend C

如果你写过 CUDA，对“线程、grid、block、shared memory、global memory”这些概念一定不陌生。Ascend C 的编程模型有相似的影子，但名词和机制都不同：这里没有“线程”和“warp”，取而代之的是 **Tensor**、**TBuf**、**TQue** 和**流水线**。本章带你建立 Ascend C 的核心心智模型，作为读懂 baseline 代码和继续优化的基础。

直觉 不妨把 Ascend C 的编程模型想成一条“流水餐厅”：数据从仓库（GM）搬到操作台（UB），在操作台上加工（Vector 计算），再从操作台送回仓库（GM）。你的代码不是写每个线程干什么，而是写“这一批数据怎么搬、怎么算、怎么送”，搬运和计算之间用队列（TQue）衔接同步。

本章我们先认识 Tensor 数据视图，再理解 TPosition、TBuf、TQue 三种缓冲区机制，然后学习内存搬运 API 和 Vector 计算 API，最后把 CopyIn/Compute/CopyOut 三段式串成流水。

2 Tensor 编程模型

在 Ascend C 的编程模型中，主要操作的数据被视作一个 **Tensor**。

2.1 LocalTensor 与 GlobalTensor

Vector 指令的操作数必须位于 AIV 内的 **UB**（Unified Buffer）上。Ascend C 以 LocalTensor<T> 表示一段 UB 上的逻辑数据视图，承载实际数据类型 *T*（如 half、float）与长度信息，本身不直接分配内存，需从 TBuf 或 TQue 中获取。GlobalTensor<T> 则是 GM 上的对应视图，用于 DataCopy 等搬运 API 的源/目的。

Ascend C 也提供了单个数据的 API（如 GetValue 和 SetValue），但这两个 API 的性能通常比较糟糕，应避免在热点路径使用。

2.2 TPosition：缓冲区的物理位置

TPosition 是一个枚举，标识缓冲区在存储层级中的物理位置，常见的三种取值如下：

TPosition	位置	用途
VEGIN	与 MTE2 流水衔接的输入侧 UB	通常作为搬入队列位置
VECOUT	与 MTE3 流水衔接的输出侧 UB	通常作为搬出队列位置
VECCALC	纯 Vector 侧的 UB	不参与 TQue 跨流水线同步，常用于常驻或中转数据

直觉 不妨把三个 TPosition 想成厨房里三个不同用途的操作台：VEGIN 是“进货台”，东西刚从仓库搬来放着；VECOUT 是“出货台”，成品摆好等着送回仓库；VECCALC 是“备用台”，放常驻调料或临时中转的半成品，不参与进出的排队。

3 TBuf: 静态缓冲区

TBuf<TPosition> 是 UB 上的**静态缓冲区**，由 TPipe::InitBuffer 在 kernel 启动前按字节大小分配，生命周期与 kernel 一致。它直接持有 LocalTensor，不参与任何同步事件的插入，适合存放常驻数据（如 weight）或临时中转缓冲（如平方缓冲、规约暂存）。

```
TPipe pipe;
TBuf<TPosition::VECCALC> weightBuf;
pipe.InitBuffer(weightBuf, weightBytes);
LocalTensor<half> weight = weightBuf.Get<half>();
```

TBuf 分配的是整段 UB，你只管往里放数据，不需要关心同步，因为它不跨流水线。但正因如此，它不适合存放需要被流水线排队的输入输出数据。

4 TQue: 队列与流水

TQue<TPosition, BUFFER_NUM> 在 TBuf 之上提供**队列语义**，用四个动作描述一段 LocalTensor 在流水阶段间的所有权与同步事件转移：

1. AllocTensor: 从队列中分配一块空闲 LocalTensor，准备接收数据。
2. EnQue: 数据已写入（如搬入完成），把 LocalTensor 入队，并自动插入对应的同步事件（如 MTE2 $\rightarrow$ V）。
3. DeQue: 从队列中取出一块已就绪的 LocalTensor，供下一阶段使用，自动等待同步事件完成。
4. FreeTensor: 使用完毕，把 LocalTensor 释放回队列，供下一轮复用。

BUFFER_NUM 控制队列深度，设为 2 即为相邻 tile 的搬入与计算提供交叠空间，这正是 **Double Buffering**（双缓冲）的基础。

例 取一个 tile 的处理来看 TQue 的生命周期。设 BUFFER_NUM=2，第一个 tile 被 AllocTensor 分配后，DataCopy 把 GM 数据搬入，EnQue 入队（自动插入 MTE2 $\rightarrow$ V 同步）。此时 DeQue 取出第一个 tile 给 Vector 计算的同时，AllocTensor 已经可以分配第二个 tile 的缓冲，MTE2 开始搬入第二个 tile。这样第一块在算、第二块在搬，两条流水线交叠执行，这就是 double buffer 的核心思想。

直觉 不妨把 TQue 想成一条传送带，带上有两个槽位（BUFFER_NUM=2）。一个槽位上的料还在被加工时，另一个槽位已经在装新料了。AllocTensor 是拿空槽，EnQue 是装好料推上传送带，DeQue 是从传送带上取料加工，FreeTensor 是加工完把槽腾出来。传送带自动保证“料没搬好不能加工，没加工完不能腾槽”。

5 内存搬运类 API

5.1 DataCopy

DataCopy 用于 GM $\leftrightarrow$ UB 间对齐、连续或规则分段的数据搬运。它是最常用的搬运接口，调用形式大致为：

```
DataCopy(dstLocal, srcGlobal, dataSize);
```

5.2 DataCopyPad

DataCopyPad 进一步支持非 32B 对齐的尾块，可通过 isPad、leftPadding/rightPadding、paddingValue 控制尾部的填充值，避免越界读写或破坏对齐。DataCopyExtParams 以 blockCount/blockLen/srcStride/dstStride 描述多段搬运的几何。

32B 对齐是昇腾 NPU 的硬性要求：FP16 下 16 元素、FP32 下 8 元素为一个对齐单位。当 H 不是对齐单位的整数倍时（如 $H = 3037$ ），尾块必须用 DataCopyPad 正确处理，不能为了对齐而越界读写。

6 Vector 计算 API

6.1 元素级与逐元素运算

Vector 侧的元素级与逐元素运算包括 Cast（类型转换，可指定 RoundMode）、Add、Mul、Div、Sqrt、Rsqrt 等，通常以 dst, src0, src1, maskCount 形式调用。Duplicate<T> 用标量广播填充一段 LocalTensor。所有调用都按 32B 对齐（FP16 16 元素、FP32 8 元素）操作，非对齐尾数由 mask 或 DataCopyPad 处理。

```
Add(dstLocal, src0Local, src1Local, maskCount);
Mul(dstLocal, src0Local, src1Local, maskCount);
Cast(dstLocal, srcLocal, RoundMode::CAST_NONE, maskCount);
```

6.2 规约运算

沿一维规约的指令以 Reduce<op> 为主，可以指定规约长度等参数，规约操作包括求和、取极值等。同时 Ascend C 提供了更底层的 BlockReduce<op> 与 WholeReduce<op> 操作，用来减小 Reduce<op> 的开销。

Reduce<op> 可能存在较多的同步 flag 设置和边界检测，性能未必最优。BlockReduce<op> 与 WholeReduce<op> 在单次执行速度和吞吐效率上各有优劣，合作起来可能获得比原生 Reduce<op> 更好的规约性能。高效的规约是本算子获得高性能的关键一环，后续优化章会详细讨论。

直觉 不妨把 Reduce<op> 想成“自动挡”，开箱即用但油耗高；BlockReduce<op> 和 WholeReduce<op> 想成“手动挡”，需要你懂换挡时机，但老司机能开出更低的油耗。

7 CopyIn, Compute, CopyOut 三段式

Ascend C 的编程模型倡导把一个 tile 的处理拆成三段：

1. **CopyIn**：用 MTE2 把数据从 GM 搬入 UB，通过输入 TQue 衔接。
2. **Compute**：在 UB 上做 Vector 计算，可能涉及常驻 TBuf 中的 weight 或中间缓冲。
3. **CopyOut**：用 MTE3 把结果从 UB 写回 GM，通过输出 TQue 衔接。

这三段之间用 TQue 传递 LocalTensor 的所有权与同步事件。这样的编写范式能让编译器更好地识别依赖关系，并**自动**开启流水。

例 取 FusedAddRmsNorm 的一个 tile 来看三段式如何落地。CopyIn 阶段：xLocal = inQueX.AllocTensor(), DataCopy(xLocal, xGm), inQueX.Enqueue(xLocal), xLocal = inQueX.DeQue()。Compute 阶段：Add(rLocal, xLocal, residualLocal, mask), Mul(sqLocal, rLocal, rLocal, mask), 规约求和, Sqrt, Div, Mul 权重。CopyOut 阶段：outQueY.Enqueue(yLocal), yLocal = outQueY.DeQue(), DataCopy(yGm, yLocal), outQueY.FreeTensor(yLocal)。把 BUFFER_NUM 设为 2，相邻 tile 的 CopyIn 与 Compute 就能交叠。

8 查阅官方 API 参考

上述 API 的支持数据类型、对齐、mask、repeat、临时空间与同步的要求，以及功能和使用方法的详细描述，都在 Ascend C 官方 API 文档里有所说明。实际编写算子时，请及时查阅相关 API 文档，详见 **Ascend C API 列表**。

不同 CANN 版本的 API 细节可能有差异，请以你实验环境中 CANN 版本对应的文档为准。

9 本章你将学会

1. 解释 LocalTensor 与 GlobalTensor 的区别，指出 Vector 指令的操作数必须在 UB 上。
2. 说明 VECIN、VECOUT、VECCALC 三种 TPosition 的用途。
3. 用 TBuf 分配静态缓冲区，存放常驻数据或中转缓冲。
4. 描述 TQue 的四个动作（AllocTensor/EnQue/DeQue/FreeTensor）如何传递所有权与同步事件，说明 BUFFER_NUM 与 double buffer 的关系。
5. 使用 DataCopy 与 DataCopyPad 完成 GM $\leftrightarrow$ UB 搬运，处理 32B 对齐与尾块。
6. 调用 Cast/Add/Mul/Sqrt 等 Vector API 和 Reduce/BlockReduce/WholeReduce 规约 API。
7. 把算子组织成 CopyIn/Compute/CopyOut 三段式，让编译器自动开启流水。

10 要点速查

概念	要点
LocalTensor	UB 上的逻辑数据视图, 不直接分配内存
GlobalTensor	GM 上的数据视图, 用于 DataCopy 源/目的
TPosition	VECIN 输入侧, VECOUT 输出侧, VECCALC 纯计算侧
TBuf	静态缓冲, InitBuffer 分配, 不参与同步, 存常驻/中转
TQue	队列语义, Alloc/EnQue/DeQue/Free 四动作, 自动同步
BUFFER_NUM	队列深度, 设 2 即 double buffer
DataCopy	对齐连续搬运
DataCopyPad	非对齐尾块, DataCopyExtParams 描述多段
32B 对齐	FP16 16 元素, FP32 8 元素为一个对齐单位

概念	要点
Vector API	Cast/Add/Mul/Div/Sqrt/Rsqrt, dst, src, mask 形式
规约	Reduce<op> 自动挡, BlockReduce/WholeReduce 手动挡
三段式	CopyIn (MTE2) → Compute (V) → CopyOut (MTE3)

11 小结

本章从“CUDA 经验如何迁移到 Ascend C”出发，介绍了 LocalTensor/GlobalTensor 数据视图与三种 TPosition，用 TBuf 分配静态常驻缓冲，用 TQue 的四个动作传递所有权与同步事件并支撑 double buffer。我们学习了 DataCopy/DataCopyPad 搬运 API 和 Cast/Add/Mul/Reduce 等 Vector 计算 API，最后把算子组织成 CopyIn/Compute/CopyOut 三段式，让编译器自动开启流水。掌握这套编程模型后，你就能读懂 baseline 代码，并为后续优化打下基础。

讲义基于 HPC101 2025 Lab3.5 实验内容编写